

LOMA: Fast Auto-Scheduling on DNN Accelerators through Loop-Order-based Memory Allocation

Arne Symons, Linyan Mei, Marian Verhelst
MICAS, Department of Electrical Engineering, KU Leuven, Belgium

Abstract—The scheduling or temporal mapping of a neural network (NN) on a given hardware (HW) accelerator strongly impacts its execution energy and latency. Unfortunately the mapping space is huge and varies a lot in function of the NN-HW combination. Many design space exploration (DSE) frameworks aim at automatically exploring this vast mapping space. Yet, SotA frameworks suffer from being slow (e.g. exhaustive search), inflexible across a wide range of HW architectures (e.g. no support for uneven mapping), or can't guarantee global optimality (e.g. from relying on user-defined constraints or on random sampling). Moreover, existing frameworks are typically unable to predict the required CPU run-time and peak CPU memory requirements in advance and as such are unable to trade-off search time with optimality in a deterministic manner. This work proposes LOMA, a fast auto-scheduling methodology through loop-order-based memory allocation, which overcomes above bottlenecks. LOMA's capabilities are demonstrated at scale in finding the optimal schedule of complete MobileNetV3, resp. NASNet.

I. INTRODUCTION

Modern deep neural networks (DNNs) demonstrate large variability in their layer dimensions. Some networks are geared towards mobile applications, while others are designed for use on industrial-grade servers. At the same time, tailored DNN accelerators appear in the processor landscape.

This creates a need for fast and efficient scheduling, also called mapping, of various sized DNNs onto a wide range of accelerator architectures. Previous research has shown the importance of mapping optimization [1], which impacts energy consumption and throughput up to orders of magnitude.

The mapping search space is huge, and as a result, many design space exploration (DSE) frameworks have arisen, trying to automate the search for the best NN-HW mapping. To keep the search time within reasonable bounds, they typically rely on user-provided constraints [2], [3], [4], on subsampling the search space [5], [6], on probabilistic methods such as simulated annealing [7] or Bayesian optimization [8], and/or on machine learning approaches such as reinforcement learning [9] or genetic algorithm [10]. Yet, all these techniques can't provide a guarantee on finding the optimal mappings.

This paper proposes an alternative, systematic way to schedule DNN workloads on NN accelerators within limited CPU resources and without losing optimality. The proposed Loop-Order-based Memory Allocation (LOMA) method exploits the nature of the nested for-loops of a DNN layer to optimally order the loops temporally and assign each loop to the best level in the memory hierarchy (under a given spatial dataflow). Fig. 1 shows the inputs and outputs of LOMA and its operational principles, bringing following main benefits: 1.) Global

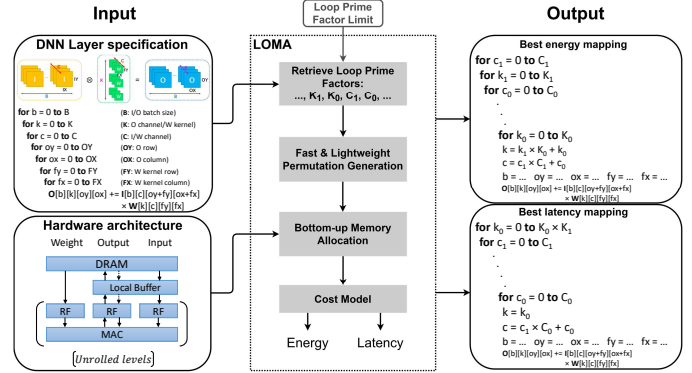


Fig. 1. Overview of the LOMA method. Inputs consist of the DNN layer spec. and the HW architecture (memory hierarchy shown). The output can differ greatly in loop ordering/splitting/allocation for optimal energy/latency.

optimality guarantee, due to its deterministic search without user-defined constraints, random sampling or probabilistic models to guide the search procedure. 2.) Support for fast auto-scheduling using both even and uneven mapping schemes [4]. 3.) Capability to trade-off search time with mapping optimality in a controlled manner by tuning the number of loop prime factors. 4.) Predictable CPU run-time and memory usage.

This paper will subsequently introduce important background knowledge on the topic of DNN accelerators, as well as discuss the related SotA in Section II. Next, the LOMA scheduling approach is detailed in Section III. Finally, Section IV will show the result of LOMA experiments and benchmarking with the SotA. Section V concludes the paper.

II. BACKGROUND AND RELATED WORK

A. DNN Layers and Loop Prime Factors

Fig. 1 (top left) shows the general structure of a CNN layer with the loop index notations used in this paper. The dimensions of the seven nested for-loops vary from layer to layer, and strongly influence data reuse for the three different operands (Input I, Weight W, Output O). It is important to note that these for-loops can be categorized as either being “irrelevant” or “relevant” to a certain operand. “Irrelevant” loops do not have an index according to the considered for-loop (e.g. B for the W), indicating data-reuse opportunities, which will be exploited later to maximize scheduling efficiency (section III-D). Alternatively, “relevant” loops appear in the operand's index, indicating required data size for that operand.

Optimizing scheduling involves the splitting of for-loops into multiple smaller for-loops, which can subsequently be reordered. This can be seen in the outputs of Fig. 1 (right).

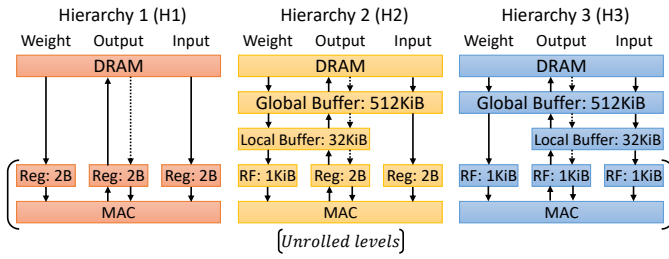


Fig. 2. Three different memory hierarchies of varying complexity. Different levels in the hierarchy can be shared by different operands (Input/Weight/Output) and can vary in size. The spatially unrolled levels are indicated in parentheses.

At the extreme, each loop dimension can be factorized into its prime factors, here also called “Loop Prime Factors” (LPFs). Depending on the loop dimensions, this greatly increases the amount of possible loop permutations and hence the amount of schedules to be assessed. These LPFs are the fundamental building blocks of LOMA’s scheduling strategy.

B. Hardware Accelerators

DNN accelerators are comprised of three key elements:

1) *Processing elements (PEs)*: consist of Multiply Accumulate (MAC) logic and optionally a small local register/register file to hold data. These PEs are typically organized into a multi-dimensional array.

2) *The memory hierarchy*: consists of one or more memory levels to locally buffer I/W/O data for data reuse in order to limit energy and latency bottlenecks from off-chip communication. The hierarchy can be a mixture of memory levels shared between operands, as well as private operand memory, as shown in Fig. 2. It is the scheduler’s task to use the memory hierarchy as efficiently as possible. In a traditional “even memory mappings”, the scheduler allocates a memory level to each for-loop, hence storing the data of I, W and O of that for-loop within the same memory level.

More recently, “uneven memory mappings” [4] allow for a fully flexible mapping in which the I, W, resp. O operand data of a for-loop can be stored at different levels in the hierarchy. As a result, the scheduler has to allocate the LPFs for each operand individually to their best memory level.

3) *The interconnection topology*: between PE’s and memories impact the supported dataflows (spatial unrolling). Conventional architectures such as Eyeriss [11] unroll the scratchpads into each PE, with dedicated I, W and O links. More recent work, such as Eyeriss v2 [12], include a highly flexible network on chip, allowing for more complex data movement between memories and PE’s. LOMA is compatible with any interconnect structure and spatial unrolling.

C. State of the art

A myriad of algorithms have been proposed to find the optimal mapping of a NN layer on an accelerator. To deal with the vast mapping space, many algorithms constrain the search space: It is common to limit the allowed loop re-ordering [5] or restrict the allowed loop sizes [6], [13]. MAGNet [2] instead imposes user-defined constraints on minimal data reuse and PE utilization, and uses Bayesian optimization to

guide the search. Likewise, Timeloop [3] randomly samples a constrained mapping space for large spaces or performs an exhaustive linear search for small (after constraints) spaces. ZigZag [4] forms constraints through a user-defined minimal memory utilization on the different memories of the hierarchy. Besides restricting the search spaces, several unconstrained probabilistic algorithms are exploited, such as Bayesian optimization [8], reinforcement learning and adaptive sampling [9], genetic algorithm in GAMMA [10] and TVM (simulated annealing) [7].

Comparing results across these different approaches often proves difficult as most use different formats for the hardware resource definitions (e.g. the memory hierarchy), use different cost models to evaluate mappings and present results based on a fixed accelerator with or without support for complex uneven memory mappings.

It is important to realize that all aforementioned algorithms strongly reduce or subsample the search space, hence losing guarantees to find the globally optimal mapping. Moreover, these approaches do not consistently report CPU execution time and memory usage, or the trade-off between CPU time and optimality. This work aims to overcome both bottlenecks.

III. THE LOMA SCHEDULING APPROACH

The LOMA algorithm features are demonstrated in Fig. 3, and will be explained in detail in the following subsections.

A. Loop Prime Factor Generation

First, we extract the LPFs for all 7 loop types (B, K, C, ...) from the layer dimensions, based on the prime factorization. For this the spatially unrolled dimensions are discarded. This step yields the building blocks for the ordering generation step.

B. Fast & Lightweight Ordering Generation

To avoid generating all possible orderings for all LPFs, which can be in the range of $10^6 - 10^9$, we generate partial permutations for each loop type separately. This has two benefits: (i) it speeds up the generation and (ii) the CPU memory size required to hold all the generated permutations drastically decreases. The partial permutations are generated through a multiset, which automatically takes into account the LPF multiplicities to avoid generating equivalent permutations [14]. Finally, the resulting multisets are recombined by looping through all of them one by one across loop types, thus yielding a complete ordering. The subsequent steps are performed on this complete ordering, after which it can be discarded if the resulting energy and/or latency is sub-optimal.

C. Unique Ordering Reduction through LPF Merging

For each complete ordering achieved during the Ordering Generation step, neighbouring LPFs of the same loop type can be merged to be scheduled together. This insight follows from the “relevant”/“irrelevant” loop principle, introduced in Section II-A. If the loop type of the LPF is irrelevant to an operand, it does not require new memory fetches and can hence be allocated to any memory level, regardless of the LPF

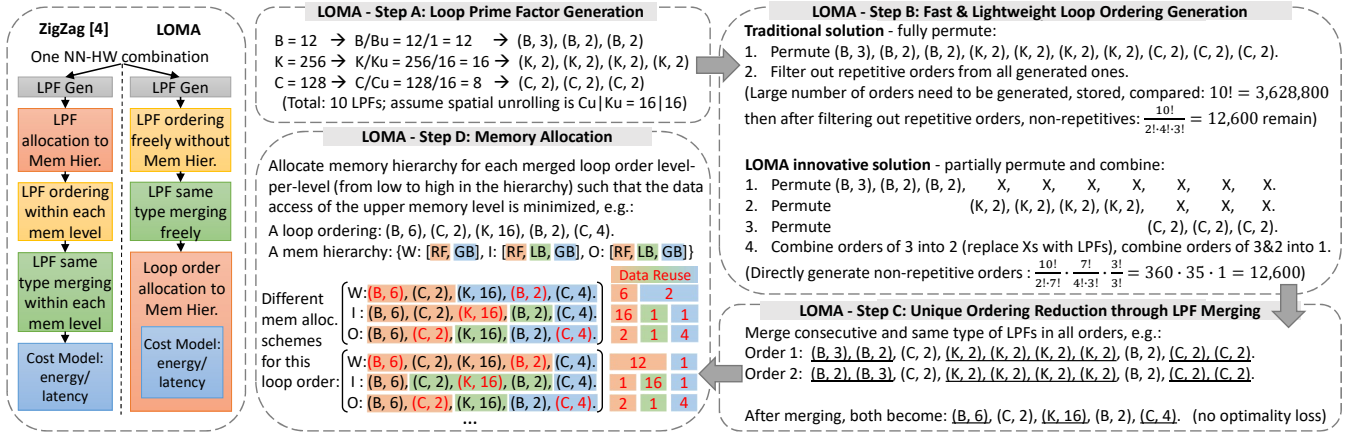


Fig. 3. Left: Comparison of ZigZag’s and LOMA’s search principles. Right: The different building blocks of the LOMA algorithm demonstrated using a NN layer with dimensions $B = 12$, $K = 256$, $C = 128$ (all other dimensions = 1) using a 16×16 PE array with $C_u|K_u$ unrolling. Each alphanumeric pair indicates a for-loop, e.g. $(B, 6)$ represents “for B in $[0, 6)$ ”.

size. If the LPF is relevant, it can occur that after merging the combined LPFs will only partially fit within a lower-level memory, requiring it to be allocated to the next memory in the hierarchy. But, as a relevant loop has no data re-use opportunities, this will lead to the same overall energy and latency, thus having no impact on optimality.

After merging, some orderings become identical and are skipped, leading to a reduction of unique orderings, and thus required search time, of 20 – 40% depending on the layer.

D. Memory Allocation

Subsequently, the scheduler will allocate all LPFs of an ordering to the memory levels. The assignment is performed by passing through all operand memories from bottom to top, always allocating as many possible LPFs of the considered ordering until a memory level is full. This allocation is split into two approaches:

1) *For non-shared memories*: the (remaining) LPFs of the memories’ operand are allocated in the order they appear in the LPF ordering until the memory is full and the next LPF cannot be assigned anymore. In this, relevant loops have to be considered to compute the required memory size for this operand (no data reuse), while irrelevant loops create reuse opportunities and can always be allocated at zero memory cost.

2) *For shared memories*: between multiple operands allocation is more complicated. Now all combinations of remaining LPFs of all operands that fit the shared memory level, have to be considered. Each combination will lead to different overall data re-use, based on the irrelevant loops encompassed in these combinations. LOMA selects as the optimal assignment the combination that: 1.) fits the memory, and 2.) incurs the highest data re-use. Although this requires more compute than the non-shared memory allocations, it does avoid an explosion of the number of scheduling options to be considered for the remaining LPFs, without giving up on optimality.

E. Trading optimality for speed

The baseline LOMA algorithm, described in previous subsections, will always find the optimal mapping for energy and

latency. This comes at the cost of evaluating every unique LPF ordering. For large networks this is, however, still infeasible within reasonable time. To cope with this, we introduce a user-defined constraint within LOMA, which limits the number of considered LPFs. When a layer has more LPFs than this value, LOMA will lump the smallest LPFs of the loop type with the most LPFs. As the experiments in Section IV will demonstrate, this allows the user to tune the algorithm to the desired runtime, with only minimal impact on optimality. This heuristic proves to work well, due to the observation that for large layers (which anyway have a large degree of scheduling freedom) the optimal mapping most often consists of fewer loops of larger iteration size.

IV. EXPERIMENTAL RESULTS AND BENCHMARKING

A. Experimental setup

The LOMA algorithm is implemented in Python and linked as an alternative temporal mapping search engine to the ZigZag DSE framework [4], which supports a wide range of even and uneven memory hierarchies and mappings. LOMA’s multiset permutations are efficiently generated in Python [14]. All experiments are run on 16 CPU cores at 3 GHz.

The validation consists of finding the optimal schedule for all 55, respectively 357 NN layers of MobileNet V3 small [15], respectively NASNet Mobile [16], across 36 different HW architectures, to show the method’s ability across a wide range of NN-HW combinations. All architectures consist of a 14×16 PE array on which OX and K are unrolled spatially. The difference between architectures are obtained by generating different memory size/sharing/bypassing combinations from a memory pool. Three such hierarchies are highlighted in Fig. 2, which will also be separately shown in the results as H1-2-3.

For all experiments, LOMA will be benchmarked in terms of search speed and optimality against ZigZag’s search engines [4]. ZigZag’s search is performed with a minimal memory utilization constraint of both 70% (fast) and 10% (slow).

For LOMA, both experiments in which the prime factor limit is kept unbounded (finding the guaranteed optimal solution), as well as experiments with user-defined LPF lumping

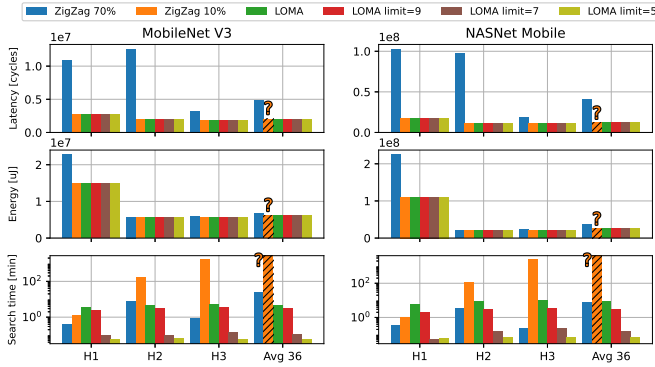


Fig. 4. Top, middle, resp. bottom graphs show best latency, best energy, resp. required search time. Horizontally, 3 individual hierarchies are shown, as well as average across all 36 hierarchies. The search is repeated using ZigZag’s search engine with a user-constrained minimal memory utilization of 70% and 10% and compared against LOMA/limited LOMA. Each bar shows the aggregated result of all 55, resp. 357 layers of MobileNet, resp. NASNet.

will be reported. This will showcase LOMA’s ability to transparently trade-off the mapping optimality vs. search time.

B. Experimental LOMA results

Fig. 4 shows the temporal mapping exploration results using the aforementioned search engines for all layers of MobileNet V3 (left) and NASNet Mobile (right). As can be seen from the search time, LOMA requires a nearly constant run-time across hierarchies, indicating that performing loop assignment for shared memories is implemented efficiently. Moreover, the optimal mappings found by LOMA are at least equally good and 8%, resp. 30% better for energy; 58%, resp. 69% better for latency on average than those found by ZigZag’s 70% search for MobileNet, resp. NASNet. ZigZag’s 10% minimal memory utilization search finds the same optimal results as LOMA for H1-2-3, however the engine fails to complete within reasonable time for other hierarchies, rendering the method impractical for some NN-HW combinations. Moreover, it is possible that for some NN-HW combination, 10% does not suffice, thus yielding sub-optimal mappings. It is clear that LOMA finds the optimal energy and latency mapping in a near-constant and predictable (but sometimes longer) search time, in contrast to the bounded search space methods exhibiting unpredictable search time.

LOMA’s search time can be traded off against minimal optimality loss in a controlled way by lumping LPFs, as discussed in Section III-E. As can be seen in Fig. 4, limiting to 9, 7, resp. 5 total LPFs for a single layer speeds up search by a factor 2x, 50x, resp. 100x, as there are exponentially fewer orderings to be evaluated. Interestingly, this greatly improved search time only rarely causes a sub-optimal mapping and if it does, with minimal impact. This can be explained by the fact that only layers with a large amount of LPFs will undergo LPF lumping, and for these layers the scheduler still has ample degrees of freedom.

We conclude that in order to speed up temporal mapping search, the golden dial to turn is limiting the number of LPFs. An additional benefit of LOMA is it’s possibility to perfectly predict the search time for a given LPF limit setting, hence

allowing to optimally tune the search space to the user’s time and computational constraints.

V. CONCLUSION

This paper presented LOMA: a fast, systematic, auto-scheduling method to find the optimal energy and latency mapping of a DNN on a HW accelerator. By combining an innovative lightweight permutation generator of all loop orderings with a bottom-up memory allocation, LOMA executes in near-constant and predictable CPU run-time with a small memory footprint. The algorithm is linked as an alternative temporal mapping search engine to the ZigZag DSE [4], allowing for complex memory hierarchies and uneven mapping of the different operands to these memories. Compared to a SotA mapping engine, LOMA finds on average 8%, resp. 30% better energy mappings; and 58%, resp. 70% better latency mappings for MobileNet V3, resp. NASNet Mobile.

Moreover, a user-defined LPF limit allows the exploration of any NN layer, regardless of it’s size, in perfectly tunable CPU run-time with minimal impact on optimality.

This work is, to our knowledge, the first of its kind, considering the required mapping search time as a key component of the algorithm’s design. In the future, LOMA’s tunable run-time can as such enable hardware-aware Neural Architecture Search, as well as full hardware architecture exploration. LOMA is open-sourced at github.com/ZigZag-Project/zigzag.

ACKNOWLEDGMENT

This research received funding from VLAIO (SPIC project), the Fund for Scientific Research Flanders (FWO-Vlaanderen), and the Flemish Government (AI Research Program).

REFERENCES

- [1] H. Kwon *et al.*, “Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach,” in *MICRO*, 2019.
- [2] R. Venkatesan *et al.*, “Magnet: A modular accelerator generator for neural networks,” in *ICCAD*, 2019.
- [3] A. Parashar *et al.*, “Timeloop: A systematic approach to dnn accelerator evaluation,” in *ISPASS*, 2019.
- [4] L. Mei *et al.*, “ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators,” *TC*, 2021.
- [5] Y. Shen *et al.*, “Maximizing cnn accelerator efficiency through resource partitioning,” in *ISCA*, 2017.
- [6] M. Song *et al.*, “Towards efficient microarchitectural design for accelerating unsupervised gan-based deep learning,” in *HPCA*, 2018.
- [7] T. Chen *et al.*, “Tvm: An automated end-to-end optimizing compiler for deep learning,” in *OSDI*, 2018.
- [8] B. Reagen *et al.*, “A case for efficient accelerator design space exploration via bayesian optimization,” in *ISLPED*, 2017.
- [9] B. H. Ahn *et al.*, “Reinforcement learning and adaptive sampling for optimized dnn compilation,” *arXiv preprint arXiv:1905.12799*, 2019.
- [10] S.-C. Kao and T. Krishna, “Gamma: automating the hw mapping of dnn models on accelerators via genetic algorithm,” in *ICCAD*, 2020.
- [11] Y.-H. Chen *et al.*, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *JSSC*, 2016.
- [12] Y.-H. Chen, T.-J. Yang *et al.*, “Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices,” *JETCAS*, 2019.
- [13] A. Stoutchinin *et al.*, “Optimally scheduling cnn convolutions for efficient memory access,” *arXiv preprint arXiv:1902.01492*, 2019.
- [14] E. Garrison. Python implementation of multiset permutation generation. [Online]. Available: <https://github.com/ekg/multipermute>
- [15] A. Howard *et al.*, “Searching for mobilenetv3,” in *ICCV*, 2019.
- [16] B. Zoph *et al.*, “Learning transferable architectures for scalable image recognition,” in *CVPR*, 2018.